

Programmation : python

Driss MATROUF

Maître de conférences, HDR de l'université d'Avignon

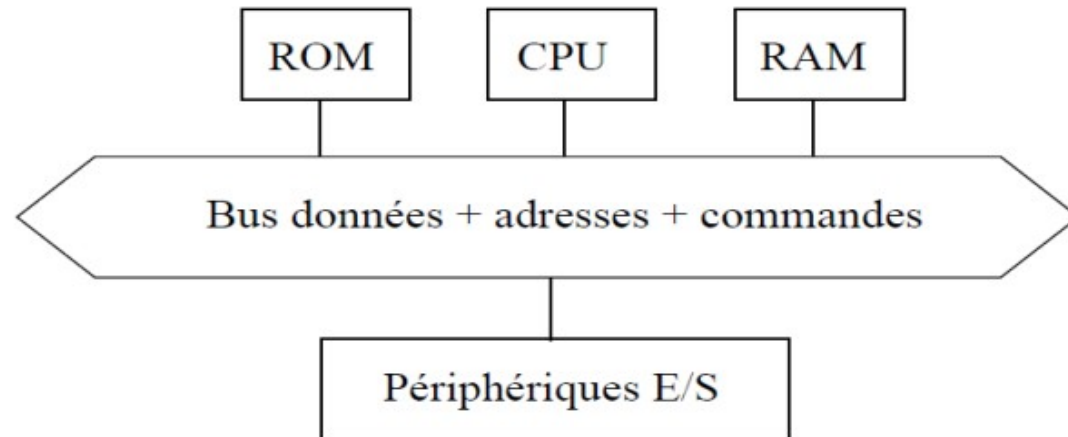
8 crédits

2 Examens Ecrits

1 Examen TP

1 Mini-projet

Structure générale



- CPU : (Central Processing Unit) exécute les instructions.
CPU n bits => les infos sont sur n bits
- ROM : Read Only Memory (non volatile)
- RAM : Random Access Memory (volatile)
- Périphériques : Disque dur (E/S), clavier(E), écran (S), lecteur cdrom (E), graveur (S).....

Execution d'un programme

- L'activité d'une machine est rythmée par le CPU qui exécute le cycle suivant :
 - chercher une instruction dans la RAM
 - exécuter
 - chercher l'instruction suivante
- Une instruction = suite de bits contenant
 - le code de l'opération (addition, multiplication ...)
 - les opérandes (arguments de l'opération)

Les différents niveaux des langages

- Le **langage machine** : chaque processeur possède un langage propre, directement exécutable par la machine. (pas portable)
- Le **langage d'assemblage** : est un codage alphanumérique du langage machine. Il est plus lisible que le langage machine. On le traduit en langage machine en utilisant un assembleur. (toujours pas portable)
- Les **langages haut niveau** : Ils sont normalisés, ils permettent le portage d'une machine à l'autre. Ils sont traduits en langage machine par un compilateur ou un interpréteur.

Exemples : C, Pascal, C++, JAVA, Python....

Production des programmes

- La **compilation** est la traduction du source en langage objet (plusieurs analyses : lexicale, syntaxique, et sémantique). La phase d'édition de liens permet de combiner les fichiers objets pour produire l'exécutable :

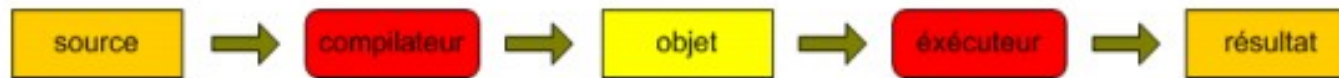


Figure 1.1 – Chaîne de compilation

- L'**interprétation** traduit au fur et à mesure les instructions en code exécutable par la machine (une certaine lenteur)



Figure 1.2 – Technique de l'interprétation

Technique de production de Python

- Technique **mixte** : l'**interprétation du bytecode compilé** : un bon compromis entre la facilité de développement et la rapidité d'exécution.
- Le bytecode est portable sur toute machine disposant d'une machine virtuelle Python
- Pour chaque module importé, Python vérifie s'il existe une version précompilée (.pyo ou .pyc) dont la date est postérieure à celle du .py



Figure 1.3 – Interprétation du bytecode compilé

Python

- Sortie en 1991
- Associer à une organisation à but non lucratif « la Python Software Foundation », créée en 2001
- Langage puissant à la fois facile à apprendre et riche en possibilités
- Un langage de programmation interprété
- Langage Open Source
- Travail interactif
- Importantes documentations en ligne
- Indentation significative
- Langage interprété rapide : interprétation du bytecode compilé
- Orienté objet
- Disponibilité de bibliothèques : plusieurs milliers de packages sont disponibles dans tous les domaines.

Algorithme et programme

- **Algorithme** : ensemble des étapes permettant d'atteindre un but donné.
- **Programme** : est la traduction d'un algorithme en un langage compilable ou interprétable.
- Module (package) : un programme peut faire appel à plusieurs modules, pour réaliser différentes tâches, en python on importe un package : **#import math** *# pour importer la package math*
#from math import cos

Premiers pas

- D'abord, il faut lancer l'interpréteur de commandes :

```
matrouf@matrouf-Latitude-E6530: ~  
matrouf@matrouf-Latitude-E6530:~$ python  
Python 2.7.12 (default, Nov 19 2016, 06:48:10)  
[GCC 5.4.0 20160609] on linux2  
Type "help", "copyright", "credits" or "license" for more information.  
>>> 
```

- Il ne comprend rien, celui la !!!

```
matrouf@matrouf-Latitude-E6530: ~  
matrouf@matrouf-Latitude-E6530:~$ python  
Python 2.7.12 (default, Nov 19 2016, 06:48:10)  
[GCC 5.4.0 20160609] on linux2  
Type "help", "copyright", "credits" or "license" for more information.  
>>> bonjour, comment ça va ?  
      File "<stdin>", line 1  
        bonjour, comment ça va ?  
              ^  
SyntaxError: invalid syntax  
>>> 
```

Les opérations et les nombres

La division entière de 13 par 4

```
matrouf@matrouf-Latitude-E6530: ~  
>>>  
>>> 3+7  
10  
>>> 3.0+7  
10.0  
>>> 10/3  
3  
>>> 10.0/3  
3.3333333333333335  
>>> 12//3  
4  
>>> 13//3  
4  
>>> 13%3  
1  
>>> 14%3  
2  
>>> 10/5  
2  
>>> 10 / 5  
2  
>>> 10%6  
4  
>>>
```

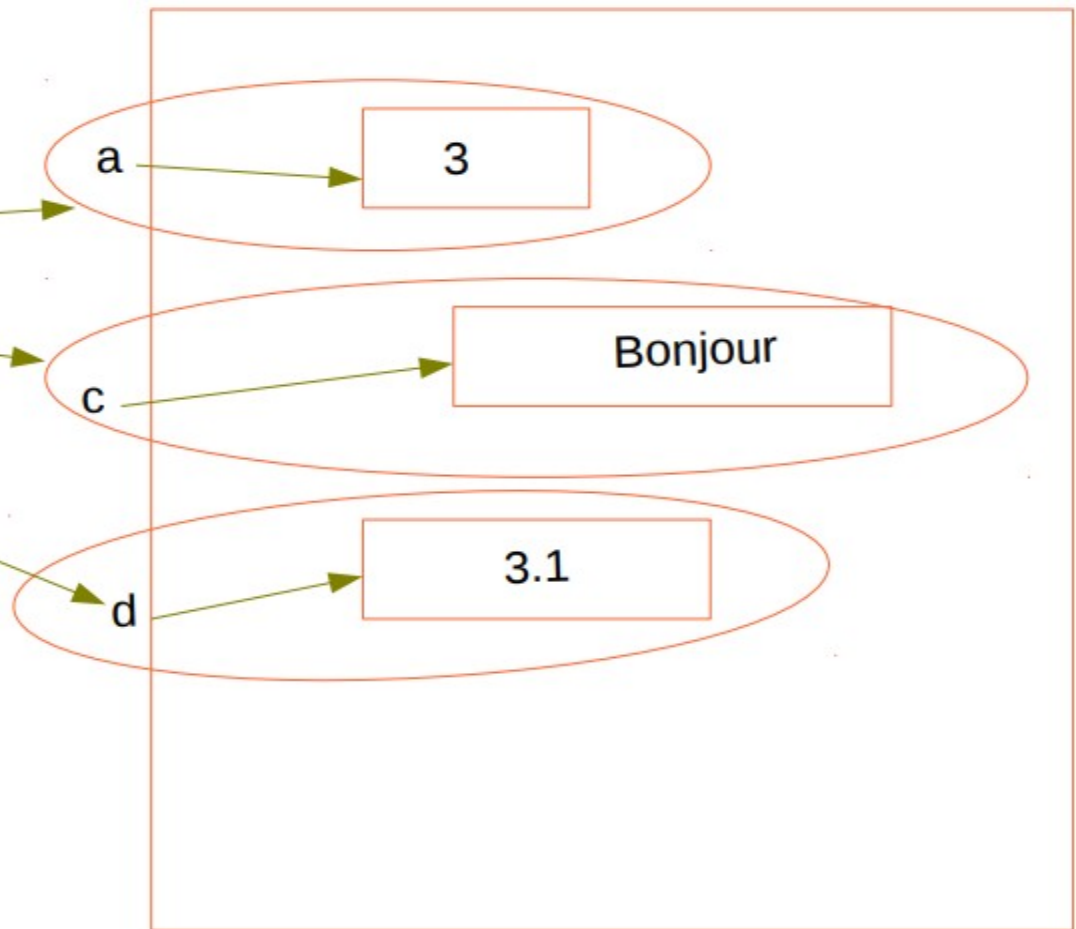
Le reste de la division de 13 par 3

Variable

- C'est une nécessité absolue

La mémoire : RAM

```
matrouf@matrouf-Latitud
>>>
>>>
>>> a=3
>>> c="bonjour"
>>> d=3.1
>>> 
```



Variable

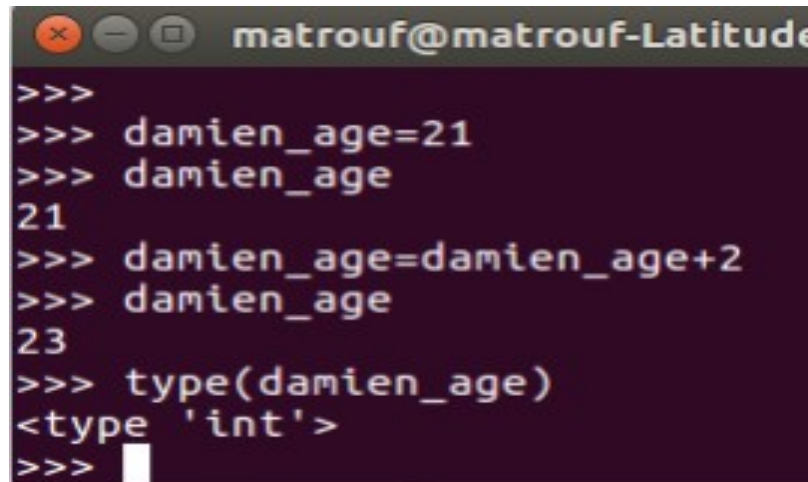
```
matrouf@matrouf-Latitude-E6530: ~  
>>>  
>>> damien_age=21  
>>> damien_age  
21  
>>> damien_age=damien_age+2  
>>> damien_age  
23  
>>> 
```

Mots réservés

and	del	from	none	true
as	elif	global	nonlocal	try
assert	else	if	not	while
break	except	import	or	with
class	false	in	pass	yield
continue	finally	is	raise	
def	for	lambda	return	

Les types des variables

- >> a=3 # on vient de créer une variable de type entier (int)
- >> a=3.1 # on vient de créer une variable de type réel (float)
- >> a="bonjour" # on vient de créer une variable de type chaîne de caractères (str)
- Pour connaître le type d'une variable, on fait : type(a)



```
matrouf@matrouf-Latitude
>>>
>>> damien_age=21
>>> damien_age
21
>>> damien_age=damien_age+2
>>> damien_age
23
>>> type(damien_age)
<type 'int'>
>>>
```

Le type booléen

- Une variable de type **booléen (bool)** ne peut prendre que deux valeur : True ou False
- Opérateurs de comparaison : `==`, `!=`, `>`, `≥`, `<` et `≤` permettent de construire les expression logique de base. Une expression logique a une valeur de type booléen.
- Une valeur booléen peut être stockée dans une variable de type booléen

```
>> a= 2>3
```

```
>> print(a)
```

```
False
```

```
>> damien_age=20
```

```
>> b=(damien_age < 30)
```

```
>> b
```

```
True
```

Expressions booléennes

- Expressions logiques
- Résultat : vrai (**True**) ou faux (**False**)
- Utilisées en commandes conditionnelles et dans les boucles
- Analyser l'état d'un calcul afin de choisir le prochain branchement
- Opérateurs utilisés
 - Relationnels : **>** **<** **==** **!=** **>=** **<=**
 - Booléens : **and**, **or**, **not**
- Evaluation
 - Expression évaluée de gauche à droite
 - Selon le résultat (vrai ou faux) qui peut être déterminé sans évaluer ce qui reste

Print

```
>> print(5) # nous utiliserons python3, donc les () sont nécessaires
```

```
>> a=8
```

```
>> b=2
```

```
>> print(a) # affiche la valeur de la variable a
```

```
8
```

```
>> print(a,b)
```

```
8 2
```

```
>> print(a,"puis",b)
```

```
8 puis 2
```


input

- Pour faire une saisie à partir du clavier

```
a=input()
```

- Dans python3 a est de type str (chaîne de caractères)
- Afin de saisir un entier (par exemple) :

```
a=int(input())
```

- Afin de saisir un réel (par exemple) :

```
a=float(input())
```

- On peut aussi afficher un message avant de réaliser la saisie :

```
a=int(input("Entrez une valeur entière"))
```

Instruction conditionnelle

Formats :

- **if** *expression*:
 commandes
 Exécute les commandes si l'expression est vraie
- **if** *expression*:
 commandes1
 Exécute les commandes 1 si l'expression est vraie.
 else:
 commandes2
 Autrement, il exécute les commandes 2
- **if** *expression1*:
 commandes1
 Exécute les commandes 1 si l'expression1 est vraie
 elif *expression2*:
 commandes2
 Autrement, il teste l'expression2 et exécute les commandes 2 si elle est vraie
 else:
 commandes(N)
 Autrement, il exécute les commandes N

Exemple : if

```
a=int(input("Entrez une valeur :"))
```

```
if a < 0 :
```

```
    print (a," est négative")
```

```
print("Merci")
```

Exécution 1 :

Entrez une valeur:2

Merci

Exécution 2 :

Entrer une valeur : -2

-2 est négative

Merci !

Un autre exemple

```
a=int(input("Entrer une valeur :"))
```

```
if(a<0) :
```

```
    print(a," est négative")
```

```
else :
```

```
    print(a," est positive ou nulle")
```

```
print("Merci")
```

Exécution 1 :

Entrer une valeur : -2

-2 est négative

Merci !

Ifelse if

```
a=int(input("Entrer une valeur"))
```

```
if a < 0 :
```

```
    print(a," est négative")
```

```
elif a==0 :
```

```
    print(a," est nul")
```

```
else :
```

```
    print(a, " est positif")
```

Exécution1 :

Entrer une valeur:0

0 est nul

Retour aux booléens

Ecrire un programme qui saisit une valeur comprise entre 10 et 20, qui la multiplie par 2 et qui affiche le résultat.

```
x=int(input("Entrez unz valeur comprise entre 10 et 20"))
```

```
If x>=10 and x<=20 :
```

```
    print("Le résultat est :",x*2)
```

```
else :
```

```
    print("LA valeur doit être comprise entre 10 et 20")
```

```
print("Au revoir")
```

Test - booléen

Ecrire un programme qui demande à l'utilisateur deux entiers et qui dit si l'un est multiple de l'autre....

```
B=False
```

```
x=int(input("....."))
```

```
y=int(input("....."))
```

```
if(x>=y) :
```

```
    if(x%y)==0 :
```

```
        B=True
```

```
Else :
```

```
    if(y%x)==0 :
```

```
        B=True
```

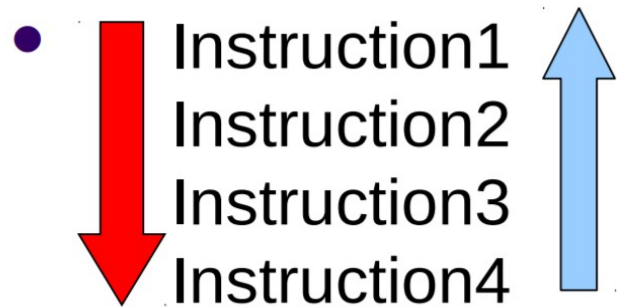
```
If (B) :
```

```
    print("OUI")
```

```
Else :
```

```
    print("NON")
```

Les boucles



- While, for
- L'ordinateur lit les instructions de haut en bas
- Puis, une fois arrivé à la fin de la boucle, il repart à la première instruction
- Il recommence alors à lire les instructions de haut en bas...
- ... Et il repart au début de la boucle.

while

while expression_logique :

Instruction1

Instruction2

.....

Exemple :

i=0

while i<6 :

print("Bonjour",i)

i=i+1

Bonjour 0

Bonjour 1

Bonjour 2

Bonjour 3

Bonjour 4

Bonjour 5

Boucle infinie

```
i=0  
while i<6 :  
    print("Bonjour",i)
```

```
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
Bonjour 0  
.....
```

Exo1

Ecrire un programme qui demande à l'utilisateur un entier entre 2 et 9 et qui affiche sa table de multiplication.

```
n=int(input("Entrez un entier compris entre 2 et 9"))  
while (n<2 or n>9):  
    n=int(input("Entrez un entier compris entre 2 et 9"))
```

```
i=1  
while (i<10):  
    print(i,"x",n,"=",i*n)  
    i=i+1
```

```
Entrez un entier compris entre 2 et 9 : 11  
Entrez un entier compris entre 2 et 9 : 7  
1x7=7  
2x7=14  
3x7=21  
4x7=28  
5x7=35  
6x7=42  
7x7=49  
8x7=56  
9x7=63
```

Encore while

Ecrire un programme qui demande à l'utilisateur un entier n et qui affiche 2^n

```
n=int(input("Entrez un entier > 0"))  
while (n<=0):  
    n=int(input("Entrez un entier >0"))  
  
i=0  
P=1  
  
while(i<n)  
    P=P*2  
    i=i+1  
  
print("le résultat est :",P)
```

Un autre exemple

Ecrire un programme qui demande à l'utilisateur un entier n et qui affiche un carré d'étoiles de taille n :

```
n=int(input ("Entrer un entier :"))
```

```
i=0
```

```
while(i<n) :
```

```
    j=0
```

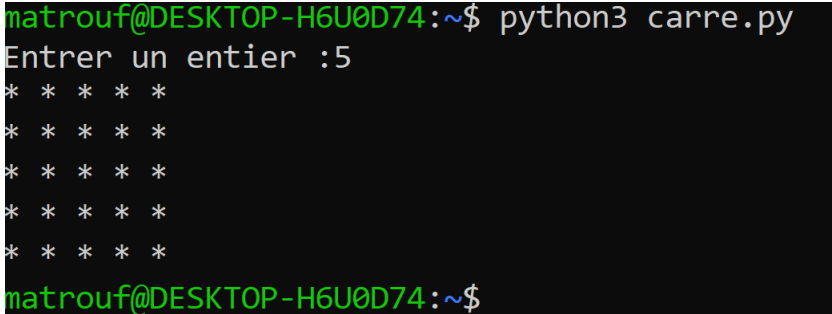
```
    while(j<n) :
```

```
        print("* ",end="")
```

```
        j=j+1
```

```
    print("\n",end="")
```

```
    i = i + 1
```



```
matrouf@DESKTOP-H6U0D74:~$ python3 carre.py
Entrer un entier :5
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
matrouf@DESKTOP-H6U0D74:~$
```

Boucles avec FOR

```
for i in H:  
    traiter i  
    .....  
# sortie de la boucle
```

H peut être par exemple une liste : [1,2,3,4,5]

Dans ce cas H est de type list

La boucle devient :

```
for i in [1,2,3,4,5]:  
    print(i)
```

H peut être aussi range(d,f,p) : de d à f-1 par pas de p

H peut être aussi range(f) : de 0 à f-1 par pas de 1 (0,1,2,.....,f-1)

```
For i in range(1,6):  
    print(i)
```

FOR

```
for i in xrange (7): # i va de 0 à 6  
    print i
```

0
1
2
3
4
5
6



```
For i in xrange (0,10,2) : # va de 0 à 9 avec un pas de 2  
    print i
```

0
2
4
6
8

For à la place du while

Ecrire un programme qui demande à l'utilisateur un entier entre 2 et 9 et qui affiche sa table de multiplication.

```
n=int(input("Entrez un entier compris entre 2 et 9"))  
while (n<2 or n>9):  
    n=int(input("Entrez un entier compris entre 2 et 9"))  
  
for i in range(1,10):  
    print(i,"x",n,"=",i*n)  
    i=i+1
```


La moyenne de valeurs saisies

Ecrire un programme qui saisit une suite de valeurs réelles et qui affiche la moyenne. La saisie se termine quand l'utilisateur saisit une valeur négative.

```
x=int(input("Entrez une valeur"))
moyenne=0.0
compteur=0
while(x>0):
    moyenne = moyenne + x
    compteur = compteur + 1
    x=int(input("Entrez une valeur"))
if(compteur > 0):
    moyenne=moyenne/compteur
    print("La moyenne est :",moyenne)
else:
    print("La moyenne n'est pas calculée")
```

La moyenne, voulez vous continuer ?

Même question que dans le slide précédent mais la saisie doit s'arrêter quand l'utilisateur répond non à la question : d'autres valeurs ?

```
moyenne=0.0 ; compteur=0 ; continuer=True
```

```
while(continuer) :
```

```
    x=float(input("Entrez une valeur"))
```

```
    moyenne = moyenne + x
```

```
    compteur = compteur + 1
```

```
    rep=input("D'autres valeurs?")
```

```
    if(rep=="non") : continuer=false
```

```
if(compteur > 0) :
```

```
    moyenne=moyenne/compteur
```

```
    print("La moyenne est :",moyenne)
```

```
else :
```

```
    print("La moyenne n'est pas calculée")
```

Break dans une boucle

Même question que dans le slide précédent mais la saisie doit s'arrêter quand l'utilisateur répond non à la question : d'autres valeurs ?

```
moyenne=0.0 ; compteur=0
```

```
while(True):
```

```
    x=float(input("Entrez une valeur"))
```

```
    moyenne = moyenne + x
```

```
    compteur = compteur + 1
```

```
    rep=input("D'autres valeurs?")
```

```
    if(rep=="non"): break
```

```
if(compteur > 0):
```

```
    moyenne=moyenne/compteur
```

```
    print("La moyenne est :",moyenne)
```

```
else :
```

```
    print("La moyenne n'est pas calculée")
```

Les listes

- Les listes sont des objets pouvant contenir d'autres objets.
- Les objets contenus dans une liste peuvent être de types différents : entiers, flottants, chaînes de caractères (str) ...
- L'implémentation de cet objet est cachée
- Accessible via son utilisation :

création, parcours, ajout, suppression

```
1  >>> ma_liste = list() # On crée une liste vide
2  >>> type(ma_liste)
3  <class 'list'>
4  >>> ma_liste
5  []
6  >>>
```

Les listes

```
1 >>> ma_liste = [] # On crée une liste vide
2 >>>
```

```
1 >>> ma_liste = [1, 2, 3, 4, 5] # Une liste avec cinq objets
2 >>> print(ma_liste)
3 [1, 2, 3, 4, 5]
4 >>>
```

Une liste peut contenir des objets de types différents

```
1 >>> ma_liste = [1, 3.5, "une chaine", []]
2 >>>
```

Comment accéder aux éléments d'une liste

```
1 >>> ma_liste = ['c', 'f', 'm']
2 >>> ma_liste[0] # On accède au premier élément de la liste
3 'c'
4 >>> ma_liste[2] # Troisième élément
5 'm'
6 >>> ma_liste[1] = 'Z' # On remplace 'f' par 'Z'
7 >>> ma_liste
8 ['c', 'Z', 'm']
9 >>>
```

- + Les listes sont des objets modifiables, elles sont dites *mutable*
- + Ajouter un élément dans une liste :

```
1 >>> ma_liste = [1, 2, 3]
2 >>> ma_liste.append(56) # On ajoute 56 à la fin de la liste
3 >>> ma_liste
4 [1, 2, 3, 56]
5 >>>
```

Attention une liste n'est pas un tableau (C,C++)

Tableau (C,C++) en RAM :



Une suite contiguës d'octets

Une liste en python :

On ne sait pas et on a pas besoin de savoir, on peut l'imaginer comme une suite de cases pour pouvoir les manipuler....

Les listes sont des objets modifiable

La plupart des fonctions appliquées à une liste ne renvoie rien, mais modifie la liste elle même.

```
10 ... liste1 = [1, 5.5, 18]
11 >>> liste2 = liste1.append(-15) # On ajoute -15 à liste1
12 >>> liste1                      # On affiche liste1
13 [1, 5.5, 18, -15]
14 >>> # Cette fois, l'appel de la méthode a modifié l'objet d'
    origine (liste1)
15 ... # Voyons ce que contient liste2
16 ... liste2
17 >>> # Rien ? Vérifions avec print
18 ... print(liste2)
19 None
20 >>>
```


+ Ajouter un élément à un rang donné dans une liste.

+ Concaténations de listes

```
1 >>> ma_liste = ['a', 'b', 'd', 'e']
2 >>> ma_liste.insert(2, 'c') # On insère 'c' à l'indice 2
3 >>> print(ma_liste)
4 ['a', 'b', 'c', 'd', 'e']
```

```
1 >>> ma_liste1 = [3, 4, 5]
2 >>> ma_liste2 = [8, 9, 10]
3 >>> ma_liste1.extend(ma_liste2) # On insère ma_liste2 à la fin
  de ma_liste1
4 >>> print(ma_liste1)
5 [3, 4, 5, 8, 9, 10]
6 >>> ma_liste1 = [3, 4, 5]
7 >>> ma_liste1 + ma_liste2
8 [3, 4, 5, 8, 9, 10]
9 >>> ma_liste1 += ma_liste2 # Identique à extend
10 >>> print(ma_liste1)
11 [3, 4, 5, 8, 9, 10]
12 >>>
```

Suppression d'un élément d'une liste

```
1 >>> ma_liste = [-5, -2, 1, 4, 7, 10]
2 >>> del ma_liste[0] # On supprime le premier élément de la
    liste
3 >>> ma_liste
4 [-2, 1, 4, 7, 10]
5 >>> del ma_liste[2] # On supprime le troisième élément de la
    liste
6 >>> ma_liste
7 [-2, 1, 7, 10]
8 >>>
```

```
1 >>> ma_liste = [31, 32, 33, 34, 35]
2 >>> ma_liste.remove(32)
3 >>> ma_liste
4 [31, 33, 34, 35]
5 >>>
```

Parcours de listes

Une fonction importante, elle retourne la taille d'une liste, c'est-à-dire le nombre d'éléments de la liste :

`L=[1,2,3,8]`

`print(len(L))`

```
1  >>> ma_liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
2  >>> i = 0 # Notre indice pour la boucle while
3  >>> while i < len(ma_liste):
4  ...     print(ma_liste[i])
5  ...     i += 1 # On incrémente i, ne pas oublier !
6  ...
7  a
8  b
9  c
10 d
11 e
12 f
13 g
14 h
15 >>> # Cette méthode est cependant préférable
16 ... for elt in ma_liste: # elt va prendre les valeurs
    successives des éléments de ma_liste
17 ...     print(elt)
18 ...
19 a
20 b
21 c
22 d
23 e
24 f
25 g
26 h
27 >>>
```